Problem Set Solutions: Seminar 07

Amortized Analysis and Sparse Table
MIT-style solution format

Algorithms and Data Structures I

June 29, 2026

Course:	Algorithms and Data Structures I
Topic:	Amortized analysis, potentials, sparse table, disjoint sparse table
Style:	Full problem-set solutions with algorithms, proofs, and complexity bounds

Why this matters. Amortized analysis is the mathematical language for algorithms that occasionally perform expensive repairs but are cheap on average over a sequence. Sparse tables are the opposite extreme: all work is pushed into preprocessing so that queries become constant-time. This sheet connects both themes: we use potentials to prove termination and linear total work, then use algebraic structure–idempotence and associativity–to decide which range queries admit $O(1)$ answers.

Contents

Conventions. Arrays are written 1-indexed unless an individual construction is cleaner in 0-indexed notation. For amortized analysis, the amortized cost is

$$\widehat{c}_i = c_i + \Phi_i - \Phi_{i-1},$$

where c_i is the actual cost and Φ is a nonnegative potential. Logs are base 2.

Problem 1. Coin exchange with strictly smaller denominations

A businessman has coins of finitely many denominations. Each day he gives one coin to a demon and receives an arbitrary finite set of coins of smaller denominations. Prove that eventually he cannot make a move.

Solution.

Idea. The number of coins may increase, so ordinary "total number of coins" is not a useful potential. The correct potential is lexicographic: the most expensive remaining denomination dominates every possible change among smaller denominations.

Correctness. Let the denominations be ordered as

$$d_1 < d_2 < \cdots < d_k.$$

A state is described by the vector

$$(c_k, c_{k-1}, \ldots, c_1),$$

where c_i is the number of coins of denomination d_i. Compare such vectors lexicographically from left to right, that is, first by the number of largest coins, then by the number of second largest coins, and so on.

If the businessman exchanges one coin of denomination d_t, then all coordinates corresponding to denominations larger than d_t remain unchanged, the coordinate c_t decreases by 1, and only smaller-denomination coordinates may increase. Therefore the vector strictly decreases in lexicographic order.

The set $\mathbb{N}^k$ with lexicographic order from highest denomination to lowest is well-founded: there is no infinite strictly decreasing sequence. Indeed, the first coordinate can decrease only finitely many times; between two such decreases, the second coordinate can decrease only finitely many times; continuing inductively over the finite number of coordinates rules out an infinite process.

Thus infinitely many exchanges are impossible. Eventually no exchange can be made. Since every allowed exchange requires an existing coin and produces only smaller coins, the only final possibility is that no coin is left that can be exchanged further; in particular, if the smallest denomination cannot be exchanged, all remaining coins must have that smallest denomination.

Running time. This is a termination proof, not an algorithm. The proof uses a well-founded lexicographic potential over a finite-dimensional vector of natural numbers.

Problem 2. Making all row and column sums nonnegative

Given an $n \times m$ matrix. One operation multiplies all entries of one row or one column by -1. Prove that after finitely many operations all row sums and all column sums can be made nonnegative.

Solution.

Idea. If a row has negative sum, flipping it strictly increases the total sum of all matrix entries. The same is true for a column. Since there are only finitely many sign configurations of rows and columns, strict improvement cannot continue forever.

Algorithm. Repeat the following operation while possible:

> If some row has negative sum, flip this row. Otherwise, if some column has negative sum, flip this column.

When neither such row nor such column exists, stop.

Correctness. Let

$$S = \sum_{i=1}^{n} \sum_{j=1}^{m} a_{ij}$$

be the total sum of the matrix. If row i has sum $R_i < 0$, then flipping row i changes the total sum by

$$(-R_i) - R_i = -2R_i > 0.$$

Similarly, if column j has sum $C_j < 0$, flipping column j changes the total sum by $-2C_j > 0$.

A sequence of row and column flips is fully described by which rows and columns have been flipped an odd number of times, so there are at most 2^{n+m} possible sign configurations. Since every performed operation strictly increases S, no configuration can repeat. Hence the algorithm must terminate. At termination there is no negative row sum and no negative column sum, exactly as required.

Running time. The proof gives finite termination. A naive implementation may recompute sums after flips; the problem asks for existence, so no optimized complexity bound is required.

Problem 3. Binary search on a growing array

A sorted static array supports membership queries in $O(\log n)$. Now suppose elements may be inserted. Store the set as a union of sorted arrays whose sizes are distinct powers of two, analogously to a binomial heap. Describe membership queries, insertions, worst-case insertion time, and amortized insertion time.

Solution.

Idea. The structure is a binary counter. A nonempty bucket of size 2^t corresponds to bit $t = 1$. Insertion creates a size-1 sorted array and repeatedly merges equal-size arrays, just like carrying in binary addition.

Algorithm. Maintain buckets $B_0, B_1, B_2, \ldots$, where each nonempty B_t is a sorted array of size 2^t, and at most one bucket of each size is nonempty.

Membership query. To test whether x is present, binary-search for x inside every nonempty bucket. There are at most $\lfloor \log n \rfloor + 1$ nonempty buckets, and each binary search costs $O(\log n)$ in the worst case.

Insertion. Create a one-element sorted array $C = [x]$. If B_0 is empty, store C there. Otherwise merge C with B_0 into a sorted array of size 2, clear B_0, and try to place the result into B_1. Continue while the target bucket is occupied.

Correctness. The invariant is that every nonempty bucket is sorted and bucket sizes are distinct powers of two. Merging two sorted arrays of the same size produces a sorted array of twice the size. Therefore after every carry step the resulting array has the correct power-of-two size, remains sorted, and represents exactly the union of the two previous arrays. When the carry chain stops, all bucket sizes are again distinct. Hence the structure stores exactly all inserted elements.

For membership, the stored set is the disjoint union of the buckets. Searching every nonempty bucket therefore finds x if and only if x belongs to the set.

Running time. A query searches $O(\log n)$ buckets and costs $O(\log^2 n)$. A single insertion may merge arrays of sizes $1, 2, 4, \ldots, 2^t$, so its worst-case time is $O(n)$ when many low buckets are occupied. Over a sequence of insertions, each element participates in at most one merge per level, hence in $O(\log n)$ total merge work per insertion on average. The amortized insertion time is $O(\log n)$.

Problem 4. Potential analysis of calls to doMagic

For an array a of length n, there are m calls of the following two types:

```
f(l,r): for i=l+1..r: if a[i] != a[i-1]: doMagic(); for i=l+1..r: a[i]=a[i-1];
g(l,r,x): for i=l..r: a[i]+=x.
```

The function doMagic does not modify the array. Prove that the total number of calls to doMagic is $O(n + m)$.

Solution.

Idea. A call to doMagic happens exactly at an internal boundary where adjacent elements are different. The operation f destroys all such internal boundaries, while operation g can create new boundaries only at the two ends of its interval.

Potential. Let

$$\Phi = \#\{i \in \{2, \ldots, n\} : a_i \neq a_{i-1}\}$$

be the number of adjacent unequal pairs. Initially $0 \leq \Phi \leq n - 1$.

Amortized analysis. Consider a call $g(l, r, x)$. All elements inside $[l, r]$ are shifted by the same value, so equalities and inequalities between adjacent elements strictly inside $[l, r]$ do not change. Only the boundary pairs $(l-1, l)$ and $(r, r+1)$ may change. Hence g can increase Φ by at most 2.

Now consider $f(l, r)$. Let d be the number of indices $i \in [l+1, r]$ such that $a_i \neq a_{i-1}$ before the first loop. Exactly d calls to doMagic are made. The second loop sets the whole interval $[l, r]$ equal to the old value of a_l, so all d internal unequal boundaries disappear. The only new unequal boundary that can be created is $(r, r+1)$, if $r < n$. Therefore

$$\Phi_{\text{after}} \leq \Phi_{\text{before}} - d + 1,$$

so

$$d \leq 1 + \Phi_{\text{before}} - \Phi_{\text{after}}.$$

Summing this inequality over all f-operations, and adding the fact that each g-operation increases the potential by at most 2, gives

$$\sum d = O(\Phi_0 + \#f + 2\#g) = O(n + m).$$

Thus the total number of doMagic calls is $O(n + m)$.

Correctness. The potential counts exactly the resource consumed by doMagic: internal unequal adjacent pairs. Operation f pays for its calls by eliminating those pairs; operation g can create only constantly many new pairs. Since the potential is always nonnegative and initially at most $n - 1$, the total number of paid events is linear in $n + m$.

Problem 5. Segment next_permutation queries and point queries in linear total time

An array $a_1, \ldots, a_n$ consists of pairwise distinct numbers. Process queries of two types: apply next_permutation to a subarray $[l, r]$, and output a_{pos}. Process q queries in $O(n + q)$ total time.

Solution.

Idea. The usual next_permutation algorithm scans the longest decreasing suffix of the chosen segment, swaps the pivot with the next larger element in that suffix, and reverses the suffix. Its cost is proportional to the length of this decreasing suffix. The amortization is by descents: reversing a decreasing suffix destroys many descents, while one application of next_permutation creates at most one new descent.

Algorithm. For an update on $[l, r]$, run the standard in-place next_permutation algorithm restricted to this segment:

1. Find the largest $i \in [l, r-1]$ such that $a_i < a_{i+1}$. If no such i exists, reverse $a_l, \ldots, a_r$ and stop.
2. Find the largest $j \in [i+1, r]$ such that $a_j > a_i$.
3. Swap a_i and a_j.
4. Reverse $a_{i+1}, \ldots, a_r$.

A point query returns the current value stored in the array cell *pos*.

Correctness. The restricted algorithm is exactly the classical NEXT_PERMUTATION algorithm applied to the sequence $a_l, a_{l+1}, \ldots, a_r$. Because all values are pairwise distinct, the rightmost ascent i is well-defined unless the segment is strictly decreasing. If the segment is strictly decreasing, it is the lexicographically largest permutation of its elements, and reversing it gives the smallest one.

Otherwise, the suffix $a_{i+1}, \ldots, a_r$ is strictly decreasing. The element a_j chosen from the right is the smallest suffix element larger than a_i. Swapping a_i with this value makes the prefix as small as possible while still increasing lexicographically. Reversing the decreasing suffix makes the remaining suffix increasing, hence lexicographically minimal. Therefore the produced segment is precisely the next lexicographic permutation of the segment.

Amortized analysis. Call an index t a descent if $a_t > a_{t+1}$. Suppose an update scans a decreasing suffix of length s. Then the suffix contains $s-1$ descents before the operation. After the operation, the reversed suffix is increasing, so these $s-1$ descents disappear. The operation can create only constantly many new descents: at most one at the new pivot boundary inside $[l, r]$, and at most two at the external boundaries $(l-1, l)$ and $(r, r+1)$ if those boundaries exist. These external descents do not affect the current scan, but they must be included in the amortized accounting.

Thus the cost of an update is $O(1 + \Delta^-)$, where Δ^- is the number of destroyed descents. Initially there are at most $n-1$ descents, and every update creates at most 3 new descents. Hence over q updates the total number of destroyed descents is $O(n+q)$, and the total time spent by all updates is $O(n+q)$. Point queries cost $O(1)$ each.

Running time. The total processing time is $O(n+q)$ and the memory usage is $O(1)$ beyond the array.

Problem 6. Correctness and exact comparison count for next_permutation

The standard NEXT_PERMUTATION procedure uses two while loops:

```
i=n-1;
while (a[i] > a[i+1]) --i;
j=n;
while (a[j] < a[i]) --j;
swap(a[i],a[j]);
reverse(a[i+1],...,a[n]);
```

Comparisons occur only in the two while loops. Prove correctness, prove that the running time is proportional to the number of comparisons, and starting from $[1, 2, \ldots, n]$, applying NEXT_PERMUTATION exactly $n! - 1$ times, find the exact number of comparisons.

Solution.

Idea. The first loop finds the longest decreasing suffix. The second loop finds the least element in that suffix that is larger than the pivot. For the exact count, classify permutations by the length of their decreasing suffix.

Correctness. Let i be the index found by the first loop. Then

$$a_{i+1} > a_{i+2} > \cdots > a_n,$$

and $a_i < a_{i+1}$. Thus the suffix $a_{i+1}, \ldots, a_n$ is the largest possible ordering of its elements. To obtain the next lexicographic permutation, we must increase the prefix as far to the right as possible; therefore the pivot is exactly i.

Inside the decreasing suffix, the rightmost element greater than a_i is the smallest element of the suffix that is greater than a_i. Swapping it with a_i gives the smallest possible increase of the prefix. After the swap, the suffix is still in decreasing order except for the removed element, and reversing it makes the suffix increasing. Therefore the suffix becomes lexicographically minimal among all suffixes compatible with the increased prefix. Hence the algorithm returns the next permutation.

The work outside the two scans is one swap and one reversal of the suffix. The reversal length is exactly the length found by the first scan, so it is bounded by a constant multiple of the number of comparisons in the first while loop. Thus the total running time is proportional to the total number of comparisons.

Answer. Let L be the length of the final decreasing suffix of the current permutation. Since we apply NEXT_PERMUTATION $n! - 1$ times starting from the identity, the last decreasing permutation is never used as input to another call, so $1 \le L \le n - 1$.

The first while loop performs exactly L comparisons: $L-1$ true comparisons inside the decreasing suffix and one final false comparison at the pivot. Therefore the total number of comparisons in the first loop is

$$S_1 = \sum_{t=1}^{n-1} \left(\frac{n!}{t!} - 1 \right) = n! \sum_{t=1}^{n-1} \frac{1}{t!} - (n-1).$$

Here $n!/t!$ counts permutations whose last t elements are decreasing; the subtraction by 1 removes the final fully decreasing permutation from the input set.

Now condition on $L = \ell$. There are

$$\frac{n!}{\ell!} - \frac{n!}{(\ell+1)!} = \frac{n!\ell}{(\ell+1)!}$$

input permutations with decreasing suffix length exactly ℓ. Among the last $\ell + 1$ elements, the pivot can have any rank $1, 2, \ldots, \ell$ among these $\ell+1$ elements except being the largest; these ranks occur equally often. If the pivot has rank s, the second loop makes exactly s comparisons. Hence the total contribution for fixed ℓ is

$$\frac{n!}{(\ell+1)!}(1 + 2 + \cdots + \ell) = \frac{n!\ell}{2\ell!}.$$

Thus

$$S_2 = \frac{n!}{2} \sum_{t=0}^{n-2} \frac{1}{t!}.$$

The exact total number of comparisons over the $n! - 1$ calls is therefore

$$\boxed{C_n = n! \sum_{t=1}^{n-1} \frac{1}{t!} - (n-1) + \frac{n!}{2} \sum_{t=0}^{n-2} \frac{1}{t!}}.$$

This is asymptotic to

$$C_n \sim \frac{3e-1}{2}\, n!.$$

Problem 7. Sparse table with the position of the minimum

Explain how, in a static array, to find not only the minimum on a segment but also a position where it occurs.

Solution.

Idea. Store the index of the minimum, not only its value. Comparing two candidate blocks then means comparing their values, with a deterministic tie-breaking rule.

Algorithm. Build a sparse table $\mathrm{st}[j][i]$ for intervals of length 2^j. Instead of storing the minimum value on $[i, i + 2^j - 1]$, store an index where this minimum is attained. The transition is

$$\mathrm{st}[j][i] = \mathrm{better}(\mathrm{st}[j-1][i], \mathrm{st}[j-1][i + 2^{j-1}]),$$

where `better` returns the index with smaller array value; in case of equality, return the smaller index if we want the leftmost minimum.

For a query $[l, r]$, set $p = \lfloor \log(r - l + 1) \rfloor$. Compare the two candidate indices

$$\mathrm{st}[p][l] \quad \text{and} \quad \mathrm{st}[p][r - 2^p + 1].$$

The better of them is an index of the minimum on $[l, r]$.

Correctness. The recurrence is correct because an interval of length 2^j is the union of two intervals of length 2^{j-1}. The minimum index of the larger interval is therefore the better of the two stored minimum indices. For a query, the two length-2^p blocks cover $[l, r]$ and are both contained in $[l, r]$. Since minimum is idempotent, overlap does not matter: the minimum over the query interval is the minimum of the two block minima. The tie-breaking rule gives a consistent occurrence position.

Running time. Preprocessing takes $O(n \log n)$ time and memory. Each query takes $O(1)$ time.

Problem 8. Append-only dynamic array with $O(1)$ range-minimum queries

Design a data structure that supports: (a) append an element to the end in amortized $O(\log n)$ time, where n is the current size; (b) return the minimum on any subarray in $O(1)$ time.

Solution.

Idea. Maintain the ordinary sparse table online. When a new element is appended, only intervals ending at the new last position become newly available.

Algorithm. Let the current array length after insertion be N. Append x as $a_N = x$ and set

$$\mathrm{st}[0][N] = x.$$

For every $j \geq 1$ with $2^j \leq N$, compute the minimum on the interval ending at N and having length 2^j:

$$\mathrm{st}[j][N - 2^j + 1] = \min\big(\mathrm{st}[j-1][N - 2^j + 1],\ \mathrm{st}[j-1][N - 2^{j-1} + 1]\big).$$

These are exactly the new sparse-table cells whose intervals include the appended element as their right endpoint.

To answer $\min(l, r)$, use the standard sparse-table query:

$$p = \lfloor \log(r - l + 1) \rfloor,$$

$$\min(l, r) = \min\big(\mathrm{st}[p][l],\ \mathrm{st}[p][r - 2^p + 1]\big).$$

Maintain the array of logarithms dynamically as well: when the length increases to N, set $\lg[N] = \lg[\lfloor N/2 \rfloor] + 1$.

Correctness. After appending a_N, every old sparse-table cell remains valid because it describes an interval not using the new element. Every interval of power-of-two length that does use the new element and has not existed before must end at N, and the algorithm computes exactly one such interval for each possible length 2^j. Therefore the sparse table remains complete and correct after each append.

The range-minimum query is correct for the same reason as in the static sparse table: two overlapping blocks of length 2^p cover $[l, r]$, and the minimum operation is associative and idempotent.

Running time. Each append computes $O(\log N)$ cells. If the rows are stored in dynamic arrays, their reallocations contribute only amortized constant overhead per pushed cell, so append costs amortized $O(\log N)$. Each range-minimum query costs $O(1)$.

Problem 9. Which operations work with sparse table?

For which range queries besides minimum can one use a sparse table?

Solution.

Answer. The standard $O(1)$ sparse-table query with two overlapping blocks works for associative and idempotent operations:

$$f(x, x) = x.$$

Examples include:

- minimum and maximum;
- greatest common divisor;
- bitwise AND and bitwise OR;
- set union and set intersection, if the represented objects support constant-time combination or if the cost of combining objects is accounted for separately.

Correctness. For a query $[l, r]$, the usual sparse table takes two blocks of equal length that cover the interval and may overlap. Associativity lets us combine precomputed block values. Idempotence makes overlap harmless, because repeated elements do not change the result.

Operations such as sum, product, matrix product, or string concatenation are associative but not idempotent. The two-overlapping-block trick would count the overlap twice, so it is not directly valid. Such operations can still be answered using a disjoint decomposition into $O(\log n)$ blocks, or using a disjoint sparse table as in Problem 10.

Running time. For associative idempotent operations, preprocessing takes $O(n \log n)$ and each query takes $O(1)$. For arbitrary associative operations with an ordinary sparse table and disjoint power-of-two decomposition, each query takes $O(\log n)$.

Problem 10. Disjoint sparse table for arbitrary associative functions

Implement a sparse-table-like structure for arbitrary associative functions f, such as sum, product, or matrix product. This is known as a disjoint sparse table.

Solution.

Idea. For arbitrary associative functions, overlapping blocks are forbidden. Instead, for every possible highest differing bit between l and r, precompute one suffix ending just before the split point and one prefix starting at the split point. Then every query $[l, r]$ splits into exactly two disjoint precomputed parts.

For clarity, use 0-indexed positions $0, 1, \ldots, n-1$.

(a) Right-going values from indices with trailing zeros. For $i \in [1, n-1]$, let $\text{zeros}(i)$ be the number of trailing zeros in the binary representation of i. For each such i, precompute

$$f(a_i, a_{i+1}, \ldots, a_r) \quad \text{for all} \quad i \le r < i + 2^{\text{zeros}(i)}.$$

The number of stored values for a fixed i is $2^{\text{zeros}(i)} = \text{lowbit}(i)$. Therefore the total size is

$$\sum_{i=1}^{n-1} \text{lowbit}(i).$$

For every $t \ge 0$, there are $O(n/2^{t+1})$ integers with exactly t trailing zeros, and each contributes 2^t. Hence the contribution of each level t is $O(n)$, and there are $O(\log n)$ possible levels. The total size is $O(n \log n)$.

(b) Left-going values from indices with trailing ones. For $i \in [0, n-1]$, let $\text{ones}(i)$ be the number of trailing ones in the binary representation of i. For each such i, precompute

$$f(a_l, a_{l+1}, \ldots, a_i) \quad \text{for all} \quad i - 2^{\text{ones}(i)} < l \le i.$$

Since $\text{ones}(i) = \text{zeros}(i+1)$, the number of values stored for i is $\text{lowbit}(i+1)$. Therefore

$$\sum_{i=0}^{n-1} 2^{\text{ones}(i)} = \sum_{i=1}^{n} \text{lowbit}(i) = O(n \log n)$$

by the same counting argument.

(c) Highest bit in $O(1)$. Precompute an array $\lg[1..n]$ by

$$\lg[1] = 0, \qquad \lg[x] = \lg[\lfloor x/2 \rfloor] + 1 \quad (x \geq 2).$$

Then $\lg[x] = \lfloor \log_2 x \rfloor$, the index of the highest set bit of x. This preprocessing takes $O(n)$ time and answers highest-bit queries in $O(1)$.

(d) Query construction. For a query $l < r$, let

$$p = \mathrm{msb}(l \oplus r),$$

where bit positions are counted from 0. The highest differing bit p is the first bit at which the binary representations of l and r diverge. Therefore

$$l = x0\ldots, \qquad r = x1\ldots$$

for the same binary prefix x. The split point is

$$m = x100\ldots0.$$

It can be computed in $O(1)$ as

$$m = r \;\&\; \sim(2^p - 1),$$

or equivalently

$$m = ((l \gg p) + 1) \ll p.$$

Then

$$l \leq m - 1 < m \leq r,$$

and the answer is the disjoint product

$$f(a_l, \ldots, a_r) = f\big(f(a_l, \ldots, a_{m-1}),\ f(a_m, \ldots, a_r)\big).$$

The left value $f(a_l, \ldots, a_{m-1})$ is present in the table from part (b), because $m - 1$ has at least p trailing ones and $l > m - 2^p$. The right value $f(a_m, \ldots, a_r)$ is present in the table from part (a), because m has at least p trailing zeros and $r < m + 2^p$.

Correctness. The split point m is exactly the boundary between the two halves determined by the highest bit on which l and r differ. Therefore $[l, m-1]$ and $[m, r]$ are disjoint and cover $[l, r]$. The two required aggregate values were precomputed by construction. Since f is associative, combining these two values in order gives exactly $f(a_l, \ldots, a_r)$. No idempotence or commutativity is required.

Running time. The preprocessing stores $O(n \log n)$ values and computes them in $O(n \log n)$ time. The array of highest bits costs $O(n)$. Each query with $l < r$ is answered in $O(1)$ time; a query with $l = r$ returns a_l directly.